

中国计量大学

本科毕业设计

（论文翻译）

YOLOv10：实时端到端目标检测

YOLOv10: Real-Time End-to-End Object Detection

学生姓名 方阳奕 学号 2200303318

学生专业 计算机 班级 22 计算机 3 班

学院 信息工程学院 指导教师 何灵敏

中国计量大学

2026 年 3 月

摘要

YOLOv10: 实时端到端目标检测

摘要: 在过去的几年里, 由于在计算成本和检测性能之间具有有效的平衡, YOLO 已成为实时目标检测领域的主要范式。研究人员已经探索了 YOLO 的架构设计、优化目标、数据增强策略等, 取得了显著进展。然而, 对后处理的非极大值抑制 (NMS) 的依赖限制了 YOLO 的端到端部署, 并对推理延迟产生不利影响。此外, YOLO 各组件的设计缺乏全面和深入的检查, 导致明显的计算冗余, 并限制了模型的能力。这使其效率不尽如人意, 同时存在显著的性能提升潜力。在本工作中, 我们旨在从后处理和模型架构两个方面进一步推动 YOLO 的性能-效率边界。为此, 我们首先提出了一致双分配的 YOLO 无 NMS 训练方法, 同时实现了竞争性性能与低推理延迟。此外, 我们引入了面向整体效率-精度的 YOLO 模型设计策略。我们从效率和精度的角度全面优化 YOLO 的各个组件, 大幅降低计算开销并增强模型能力。我们的努力成果是一代面向实时端到端目标检测的 YOLO 系列新版本, 命名为 YOLOv10。大量实验证明, YOLOv10 在各种模型规模下均实现了最先进的性能与效率。例如, 在 COCO 数据集上, 我们的 YOLOv10-S 在类似 AP 下比 RT-DETR-R18 快 1.8 倍, 同时参数量和 FLOP 分别减少 2.8 倍。与 YOLOv9-C 相比, YOLOv10-B 在相同性能下延迟降低 46%, 参数量减少 25%。代码和模型可在 <https://github.com/THU-MIG/yolov10> 获取

关键词: 学习, 神经网络, 目标检测

中图分类号: G623.58

YOLOv10: Real-Time End-to-End Object Detection

Abstract: Over the past years, YOLOs have emerged as the predominant paradigm in the field of real-time object detection owing to their effective balance between computational cost and detection performance. Researchers have explored the architectural designs, optimization objectives, data augmentation strategies, and others for YOLOs, achieving notable progress. However, the reliance on the non-maximum suppression (NMS) for post-processing hampers the end-to-end deployment of YOLOs and adversely impacts the inference latency. Besides, the design of various components in YOLOs lacks the comprehensive and thorough inspection, resulting in noticeable computational redundancy and limiting the model’s capability. It renders the suboptimal efficiency, along with considerable potential for performance improvements. In this work, we aim to further advance the performance-efficiency boundary of YOLOs from both the post-processing and the model architecture. To this end, we first present the consistent dual assignments for NMS-free training of YOLOs, which brings the competitive performance and low inference latency simultaneously. Moreover, we introduce the holistic efficiency-accuracy driven model design strategy for YOLOs. We comprehensively optimize various components of YOLOs from both the efficiency and accuracy perspectives, which greatly reduces the computational overhead and enhances the capability. The outcome of our effort is a new generation of YOLO series for real-time end-to-end object detection, dubbed YOLOv10. Extensive experiments show that YOLOv10 achieves the state-of-the-art performance and efficiency across various model scales. For example, our YOLOv10-S is $1.8\times$ faster than RT-DETR-R18 under the similar AP on COCO, meanwhile enjoying $2.8\times$ smaller number of parameters and FLOPs. Compared with YOLOv9-C, YOLOv10-B has 46% less latency and 25% fewer parameters for the same performance. Code and models are available at <https://github.com/THU-MIG/yolov10>

Keywords: Learning, neural networks, object detection

Classification: G623.58

目次

摘要	II
目次	IV
1. 引言（绪论）	1
2. 相关工作	3
3. 方法论	4
3.1. 用于无 NMS 训练的一致双重分配	4
3.2. 整体效率-精确性驱动的设计	6
4. 实验	9
4.1. 实施细节	9
4.2. 与最先进技术的比较	9
4.3. 模型分析	10
4.3.1. 针对 NMS 的免费训练分析	11
4.3.2. 用于高效驱动的设计分析	12
4.3.3. 用于精确驱动模型设计的分析	14
5. 结论	15
参考文献	16

1. 引言（绪论）

实时目标检测一直是计算机视觉领域研究的重点，旨在以低延迟准确预测图像中物体的类别和位置。它被广泛应用于各种实际场景，包括自动驾驶、机器人导航和目标跟踪等。近年来，研究人员集中精力开发基于CNN的目标检测器，以实现实时检测。其中，YOLO由于在性能和效率之间的出色平衡而受到越来越多的关注。YOLO的检测流程由两部分组成：模型前向过程和NMS后处理。然而，这两部分仍存在不足，导致精度-延迟的边界未达到最佳状态。

具体来说，YOLO在训练过程中通常采用一对多标签分配策略，即一个真实物体对应多个正样本。尽管这种方法能够带来更好的性能，但在推理阶段需要使用NMS来选择最佳的正向预测。这会降低推理速度，并使性能对NMS的超参数敏感，从而阻碍YOLO实现最佳端到端部署。一种解决此问题的方法是采用最近提出的端到端DETR架构。例如，RT-DETR提出了高效的混合编码器和最小不确定性查询选择，将DETR推进到实时应用领域。然而，仅考虑部署期间模型的前向过程时，DETR的效率相比YOLO仍有提升空间。另一种方法是探索基于CNN检测器的端到端检测，这通常利用一对一分配策略来抑制冗余预测。但它们通常会引入额外的推理开销，或在YOLO上实现次优性能。

此外，模型架构设计仍然是YOLO的一个根本性挑战，它对准确性和速度具有重要影响。为了实现更高效和更有效的模型架构，研究人员探索了不同的设计策略。在主干网络中提出了各种基本计算单元以增强特征提取能力，包括DarkNet、CSPNet、EfficientRep和ELAN等。在颈部结构方面，探索了PAN、BiC、GD和RepGFPN等，以增强多尺度特征融合。此外，还研究了模型缩放策略和重参数化技术。尽管这些努力取得了显著进展，但从效率和准确性的角度对YOLO各个组成部分的全面检查仍然缺乏。因此，YOLO中仍存在相当大的计算冗余，导致参数利用效率低下和效率不理想。此外，由此导致的模型能力受限也会导致性能较差，仍有较大的提升空间以提高准确性。

在本研究中，我们旨在解决这些问题，并进一步推动YOLO在准确性和速度方面的边界。我们针对整个检测流程中的后处理和模型结构进行优化。为此，我们首先通过提出一种适用于NMS-Free YOLO的一致双重分配策略（结合双标签分配和一致性匹配度量）来解决后处理中冗余预测的问题。这使模型在训练过程中能够获得丰富且协调的监督，同时在推理阶段无需使用NMS，从而在高效性的前提下获得具有竞争力的性能。其次，我们通过对YOLO各组件进行全面检查，提出了以整体效率-准确性为驱动力的模型架构设计策略。为了提高效率，我们提出了轻量化分类头、空间通道解耦下采样和等级引导的模块设计，以减少显性计算冗余，实现更高效的架构。为了提高准确性，我们探索了大卷积核的卷积操作，

并提出了有效的部分自注意力模块，以增强模型能力，从而在低成本下挖掘性能提升的潜力。

基于这些方法，我们成功实现了一系列新的实时端到端检测器，具有不同的模型规模，即 YOLOv10-N / S / M / B / L / X。在标准目标检测基准数据集（如 COCO）上的大量实验表明，我们的YOLOv10在各类模型规模上，在计算与精度的平衡方面可以显著超越以往的最先进模型。如图 1-1所示，我们的 YOLOv10-S / X在性能相近的情况下，比RT-DETR R18/R101分别快 $1.8\times$ / $1.3\times$ 。与YOLOv9-C相比，YOLOv10-B在保证相同性能的情况下，延迟降低了46%。此外，YOLOv10展现了高效的参数利用能力。我们的 YOLOv10-L / X 比 YOLOv8-L / X 分别高 0.3 AP和0.5 AP，同时参数量减少了 $1.8\times$ 和 $2.3\times$ 。YOLOv10-M与YOLOv9-M / YOLO-MS比较，实现了相似的AP，但参数量分别减少了23% / 31%。我们希望我们的工作能够激发该领域的进一步研究和进步。

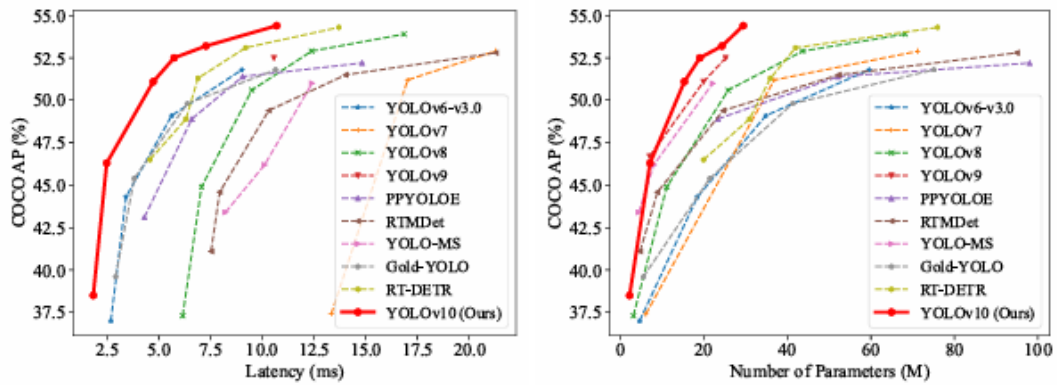


图 1-1 在延迟-准确率（左）和尺寸-准确率（右）权衡方面与其他模型的比较。我们使用官方预训练模型来测量端到端的延迟。

2. 相关工作

实时目标检测器。实时目标检测旨在低延迟下对物体进行分类和定位，这对于现实应用至关重要。近年来，已经有大量工作致力于开发高效的检测器]。其中，YOLO系列是主流方案。YOLOv1、YOLOv2和YOLOv3确立了典型的检测架构，由三部分组成，即骨干网络、颈部和检测头。YOLOv4 [2] 和 YOLOv5引入了CSPNet设计以替代 DarkNet，并结合数据增强策略、改进的PAN网络以及更多模型规模等。YOLOv6推出了用于颈部和骨干的BiC和SimCSPSPF，配合锚点辅助训练和自蒸馏策略。YOLOv7引入E-ELAN以丰富梯度流路径，并探索了多种可训练的免费优化方法。YOLOv8提出了C2f构建块以实现高效特征提取与融合。Gold-YOLO提供先进的GD机制以增强多尺度特征融合能力。YOLOv9提出GELAN改进架构，并通过PGI增强训练过程。

端到端目标检测器。端到端目标检测作为一种范式转变，摆脱了传统流水线结构，提供了更简化的架构。DETR引入了Transformer 架构，并采用匈牙利损失实现一对一匹配预测，从而消除了手工设计的组件和后处理步骤。自此之后，各种DETR变体被提出以提升其性能和效率。Deformable-DETR利用多尺度可变形注意力模块加速收敛速度。DINO将对比去噪、混合查询选择和前向两次方案整合到DETR中。RT-DETR则进一步设计了高效混合编码器，并提出了最小不确定性查询选择，以提升准确性和延迟表现。另一条实现端到端目标检测的路线是基于CNN的检测器。可学习NMS和关系网络提出另一种网络来去除检测器的重复预测。OneNet和DeFCN提出一对一匹配策略，使全卷积网络能够实现端到端目标检测。FCOSpss引入正样本选择器，用于选择预测的最优样本。

3. 方法论

3.1. 用于无 NMS 训练的一致双重分配

在训练过程中，YOLOs通常利用TAL为每个实例分配多个正样本。采用一对多分配可以提供丰富的监督信号，有助于优化并实现更优的性能。然而，这需要YOLOs依赖NMS后处理，这会导致部署时推理效率不佳。虽然先前的工作探索了一对一匹配以抑制冗余预测，但它们通常会引入额外的推理开销或产生次优性能。在本工作中，我们提出了一种适用于YOLOs的无NMS训练策略，结合双标签分配和一致匹配度量，实现了高效且具竞争力的性能。

双标签分配。与一对多分配不同，一对一匹配仅为每个真实标签分配一个预测，从而避免了NMS后处理。然而，这会导致监督信号较弱，从而导致精度和收敛速度不理想。幸运的是，这一缺陷可以通过一对多分配来弥补。为实现这一目标，我们为YOLO引入双标签分配机制，以融合两种策略的优势。

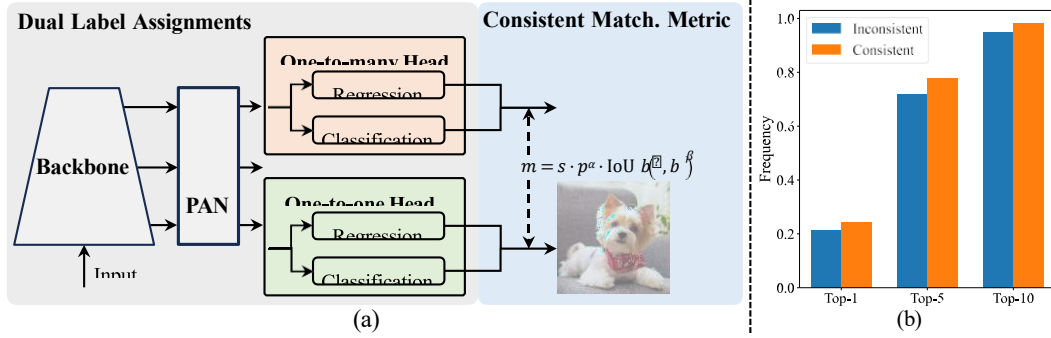


图 3.1-2 (a) 无 NMS 训练中一致的双重分配。(b) YOLOv8-S 在采用默认参数 $\alpha_0=0.5$ 和 $\beta_0=6$ 时[21], 其一对多结果在 Top-1/5/10 中的点对点分配频率。一致性条件下: $\alpha_0=0.5$; $\beta_0=6$ 。不一致性条件下: $\alpha_0=0.5$; $\beta_0=2$ 。

具体来说，如图 3.1-2 (a)所示，我们为YOLO加入了另一个一对一的头部。它保持相同的结构，并采用与原一对多分支相同的优化目标，但利用一对一匹配来获得标签分配。在训练过程中，两者头部与模型共同优化，使主干网络和颈部能够享受到一对多分配提供的丰富监督。在推理过程中，我们舍弃一对多头，仅使用一对一头进行预测。这使得YOLOs可以端到端部署，而无需增加任何额外的推理成本。此外，在一对一匹配中，我们采用了top-one选择，它以更少的额外训练时间实现了与Hungarian matching相同的性能。

一致匹配度量。在分配过程中，一对一和一对多的方法都利用度量来定量评估预测与实例之间的一致程度。为了实现对两个分支的预测感知匹配，我们采用统一的匹配度量，即

$$m(\alpha, \beta) = s \cdot p^\alpha \cdot \text{IoU}(\hat{b}, b)^\beta, \quad (1)$$

其中 p 是分类分数， $\hat{b}$ 和 b 分别表示预测框和实例框。 s 表示空间先验，用于指示预测的锚点是否位于实例内。 α 和 β 是两个重要的超参数，用于平衡语义预测任务和位置回归任务的影响。我们分别将一对多和一对一指标表示为 $m_{o2m} = m(\alpha_{o2m}, \beta_{o2m})$ 和 $m_{o2o} = m(\alpha_{o2o}, \beta_{o2o})$ 。这些指标会影响两个头部的标签分配和监督信息。

在双标签分配中，一对多分支提供的监督信号比一对一分支丰富得多。直观地说，如果我们能够将一对一分支的监督与一对多分支的监督协调起来，我们就可以将一对一分支的优化方向引导到一对多分支的优化方向上。结果是一对一分支在推理过程中可以提供更高质量的样本，从而提升性能。为此，我们首先分析两个分支之间的监督差距。由于训练过程中的随机性，我们从初始化两个分支为相同数值并产生相同预测开始进行研究，即一对一分支和一对多分支对每个预测-实例对产生相同的 p 和IoU。我们注意到，两个分支的回归目标并不冲突，因为匹配的预测共享相同的目标，而未匹配的预测会被忽略。因此，监督差异存在于不同的分类目标上。对于一个实例，我们将其与预测的最大IoU记为 u^* ，将一对多和一对一匹配的最大得分分别记为 m_{o2m}^* 和 m_{o2o}^* 。假设一对多分支产生正样本集合 Ω ，而一对一分支选择第 i 个预测，其指标为 $m_{o2o,i} = m_{o2o}^*$ ，则我们可以得到分类目标： $t_{o2m,j} = u^* \cdot \frac{m_{o2m,j}}{m_{o2m}^*} \leq u^*$ ，其中 $j \in \Omega$ ，以及 $t_{o2o,i} = u^* \cdot \frac{m_{o2o,i}}{m_{o2o}^*} = u^*$ ，用于所述的任务对齐损失。因此，两个分支之间的监督差异可以通过不同分类目标的1-Wasserstein距离来推导，即

$$A = t_{o2o,i} - I(i \in \Omega) t_{o2m,i} + \sum_{k \in \Omega \setminus \{i\}} t_{o2m,k}, \quad (2)$$

我们可以观察到，随着 $t_{o2m,i}$ 的增加，间隙会减小，即 i 在 Ω 中的排名更靠前。当 $t_{o2m,i} = u^*$ 时，即 i 是 Ω 中最优的正样本，间隙达到最小，如图 3.1-2 (a)所示。为了实现这一点，我们提出了一致匹配度量，即 $\alpha_{o2o} = r \cdot \alpha_{o2m}$ 和 $\beta_{o2o} = r \cdot \beta_{o2m}$ ，这意味着 $m_{o2o} = m_{o2m}^r$ 。因此，对于一对多头来说的最佳正样本，对一对一头也是最佳的。因此，两个头可以一致且协调地进行优化。为了简化起见，我们四者取 $r = 1$ ，即默认 $\alpha_{o2o} = \alpha_{o2m}$ 和 $\beta_{o2o} = \beta_{o2m}$ 。为了验证改进后的监督对齐效果，我们统计训练后在一对多结果的top-1 / 5 / 10 中一对一匹配对的数量。如图 3.1-2 (b)所示，在一致匹配度量下，对齐效果有所提升。欲对数学证明有更全面的理解，请参见附录。

与其他对等方法的讨论。同样，之前的工作探讨了不同的分配方式，以加速训练收敛并提升不同网络的性能。例如，H-DETR、Group-DETR和MS-DETR通过混合或多组标签分配，在原有的一对一匹配的基础上引入一对多匹配，从而改进DETR。不同的是，为了实现一对多匹配，它们通常会引入额外的查询或重复使用用于二分匹配的真实标签，或者从匹配得分中选择前几个查询，而我们采用

结合空间先验的预测感知分配。此外，LRANet采用密集分配和稀疏分配两条分支进行训练，这些都属于一对多分配，而我们采用一对多和一对一分支。DEYO在第一阶段对卷积编码器进行一对多匹配的逐步训练，在第二阶段对Transformer解码器进行一对一匹配，而我们避免使用Transformer解码器进行端到端推理。与结合双重分配的CNN检测器的工作相比，我们进一步分析了两条Head之间的监督差距，并提出了针对YOLO的一致性匹配指标，以减小理论监督差距。通过更好的监督对齐，它提升了性能，并消除了超参数调优的需求。

3.2. 整体效率-精确性驱动下的模型设计

除了后处理之外，YOLO 的模型架构在效率与准确性权衡方面也带来了巨大挑战。尽管先前的研究探索了各种设计策略，但对于 YOLO 各个组件的全面检查仍然不足。因此，模型架构存在不可忽视的计算冗余和受限的能力，这限制了其实现高效率和高性能的潜力。在这里，我们旨在从效率和准确性的角度，全面地进行 YOLO 模型设计。

以效率为驱动下的模型设计。YOLO 的组件包括干茎层、下采样层、由基本构建块组成的阶段以及头部。干茎层的计算成本很低，因此我们对其他三个部分进行以效率为驱动下的模型设计。

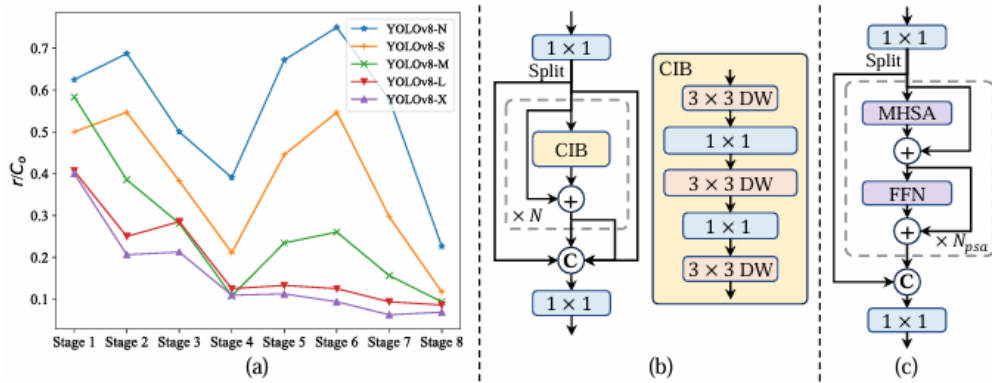


图 3.2-3 (a) YOLOv8 中各阶段和各模型的固有秩。主干网络和颈部的阶段按模型前向过程的顺序编号。数值秩 r 在 y 轴上归一化为 r/Co ，其阈值默认为 $\lambda_{\max}/2$ ，其中 Co 表示输出通道的数量， $\lambda_{\max}$ 为最大奇异值。可以观察到，深层阶段和大模型的固有秩值较低。(b) 紧凑型倒置块 (CIB)。(c) 部分自注意力模块 (PSA)。

- (1). 轻量级分类头。在YOLO中，分类头和回归头通常共享相同的架构。然而，它们在计算开销上存在显著差异。例如，在YOLOv8-S中，分类头的FLOPs和参数量（5.95G/1.51M）分别是回归（2.34G/0.64M）的2.5倍和2.4倍。然而，在分析了分类误差和回归误差的影响（见表 4.3.1-6）后，我们发现回归头对YOLO的性能更为重要。

- (2). 空间-通道解耦下采样。YOLO通常使用步幅为2的常规 3×3 标准卷积，同时实现空间下采样（从 $H \times W$ 到 $H/2 \times W/2$ ）和通道变换（从 C 到 $2C$ ）。这会引入不可忽略的计算成本 $O(\frac{9}{2}HWC^2)$ 和参数 $O(18C^2)$ 。相反，我们提出将空间缩减与通道增加操作解耦，从而实现更高效的下采样。具体来说，我们首先利用逐点卷积调节通道维度，然后使用深度卷积进行空间下采样。这将计算成本降低到 $O(2HWC^2 + \frac{9}{2}HWC)$ 并将参数量降低到 $O(2C^2 + 18C)$ 。同时，它在下采样过程中最大化信息保留，从而在降低延迟的同时实现具有竞争力的性能。
- (3). 基于秩的块设计。YOLO通常在所有阶段使用相同的基本构建块，例如YOLOv8中的瓶颈块。为了全面研究YOLO的这种同质设计，我们利用内在秩来分析每个阶段的冗余性。具体来说，我们计算每个阶段最后一个基础模块中最后一个卷积层的数值秩，该数值秩是大于阈值的奇异值的数量。图 3.2-3 (a)展示了YOLOv8的结果，表明深层阶段和大型模型更容易表现出更多冗余。这一观察表明，简单地在所有阶段应用相同的模块设计对于实现最佳的容量-效率权衡并不理想。为了解决这个问题，我们提出了一种基于秩的模块设计方案，旨在通过紧凑的架构设计降低那些被证明冗余的阶段的复杂度。我们首先提出了一种紧凑的倒置模块（CIB）结构，它采用低成本的深度卷积进行空间混合，并使用高效的逐点卷积进行通道混合，如图 3.2-3 (b)所示。它可以作为高效的基本构建模块，例如嵌入在ELAN结构中（图 3.2-3 (b)）。然后，我们提出了一种基于秩的模块分配策略，以在保持竞争力的容量的同时实现最佳效率。具体而言，给定一个模型，我们按各阶段的内在秩升序排序。我们进一步检查将领先阶段的基本模块替换为CIB后的性能变化。如果与给定模型相比没有性能下降，我们将继续替换下一阶段，否则停止这一过程。由此，我们可以在不同阶段和模型规模上实现自适应紧凑模块设计，从而在不影响性能的情况下实现更高的效率。由于篇幅限制，算法的详细内容见附录。

以准确性为驱动的设计。我们进一步探索大核卷积和自注意力机制，以实现准确性驱动的设计，旨在以最低成本下提升性能。

- (1). 大核卷积。使用大核深度卷积是扩大感受野和增强模型能力的有效方法。然而，仅在所有阶段使用大核卷积可能会污染用于检测小目标的浅层特征，同时在高分辨率阶段引入显著的I/O开销和延迟。因此，我们建议在深层阶段的 CIB 中利用大核深度卷积。具体来说，我们将CIB中第二个 3×3 深度卷积的核尺寸增大到 7×7 ，参照[39]。此外，我们采用结构重参数化技术引入另一个 3×3 深度卷积分支，以缓解优化问题，同时不会增加推理开销。此外，随着模型规模的增大，其感受野自然扩展，使用大核

卷积的优势也逐渐减小。因此，我们仅在小规模模型中采用大核卷积。

- (2). 部分自注意力 (PSA)。自注意力由于其出色的全局建模能力，已被广泛应用于各种视觉任务中。然而，它具有较高的计算复杂性和内存占用。为了解决这一问题，考虑到普遍存在的注意力头冗余，我们提出了一种高效的部分自注意力 (PSA) 模块设计，如图 3.2-3(c) 所示。具体来说，在 1×1 卷积后，我们将特征沿通道均匀划分为两部分。我们只将其中一部分输入由多头自注意力模块 (MHSA) 和前馈网络 (FFN) 组成的NPSA块中。然后将两部分拼接，并通过 1×1 卷积进行融合。此外，我们遵循的做法，将MHSA中查询和键的维度设置为值维度的一半，并用BatchNorm替换 LayerNorm，以加快推理速度。此外，PSA 仅在分辨率最低的第4阶段之后放置，避免了自注意力的二次计算复杂度带来的过高开销。通过这种方式，全局表示学习能力可以以低计算成本融合到 YOLO中，这很好地增强了模型的能力并提升了性能。

表 3.2-1 与最先进技术比较。延迟使用官方预训练模型测量。延迟 f 表示模型在前向过程中的延迟，不包括后处理。†表示 YOLOv10 使用原始一对多训练和 NMS 的结果。以下所有结果均未使用额外的高级训练技术，如知识蒸馏或 PGI，以保证公平比较。

Model	#Param.(M)	FLOPs(G)	AP ^{val} (%)	Latency(ms)	Latency ^f (ms)
YOLOv6-3.0-N [29]	4.7	11.4	37.0	2.69	1.76
Gold-YOLO-N [60]	5.6	12.1	39.6	2.92	1.82
YOLOv8-N [21]	3.2	8.7	37.3	6.16	1.77
YOLOv10-N (Ours)	2.3	6.7	38.5 / 39.5†	1.84	1.79
YOLOv6-3.0-S [29]	18.5	45.3	44.3	3.42	2.35
Gold-YOLO-S [60]	21.5	46.0	45.4	3.82	2.73
YOLO-MS-XS [8]	4.5	17.4	43.4	8.23	2.80
YOLO-MS-S [8]	8.1	31.2	46.2	10.12	4.83
YOLOv8-S [21]	11.2	28.6	44.9	7.07	2.33
YOLOv9-S [65]	7.1	26.4	46.7	-	-
RT-DETR-R18 [78]	20.0	60.0	46.5	4.58	4.49
YOLOv10-S (Ours)	7.2	21.6	46.3 / 46.8†	2.49	2.39
YOLOv6-3.0-M [29]	34.9	85.8	49.1	5.63	4.56
Gold-YOLO-M [60]	41.3	87.5	49.8	6.38	5.45
YOLO-MS [8]	22.2	80.2	51.0	12.41	7.30
YOLOv8-M [21]	25.9	78.9	50.6	9.50	5.09
YOLOv9-M [65]	20.0	76.3	51.1	-	-
RT-DETR-R34 [78]	31.0	92.0	48.9	6.32	6.21
RT-DETR-R50m [78]	36.0	100.0	51.3	6.90	6.84
YOLOv10-M (Ours)	15.4	59.1	51.1 / 51.3†	4.74	4.63
YOLOv6-3.0-L [29]	59.6	150.7	51.8	9.02	7.90
Gold-YOLO-L [60]	75.1	151.7	51.8	10.65	9.78
YOLOv9-C [65]	25.3	102.1	52.5	10.57	6.13
YOLOv10-B (Ours)	19.1	92.0	52.5 / 52.7†	5.74	5.67
YOLOv8-L [21]	43.7	165.2	52.9	12.39	8.06
RT-DETR-R50 [78]	42.0	136.0	53.1	9.20	9.07
YOLOv10-L (Ours)	24.4	120.3	53.2 / 53.4†	7.28	7.21
YOLOv8-X [21]	68.2	257.8	53.9	16.86	12.83
RT-DETR-R101 [78]	76.0	259.0	54.3	13.71	13.58
YOLOv10-X (Ours)	29.5	160.4	54.4 / 54.4†	10.70	10.60

4. 实验

4.1. 实施细节

我们选择 YOLOv8作为我们的基准模型，因为它在延迟和精度之间达到良好平衡，并且提供了多种模型尺寸。我们在无NMS的训练中采用一致的双重分配策略，并基于此进行整体的效率-精度驱动模型设计，从而产生了我们的 YOLOv10模型。YOLOv10拥有与YOLOv8相同的变体，即N/S/M/L/X。此外，我们通过简单地增加 YOLOv10-M的宽度缩放因子，得出了一个新变体 YOLOv10-B。我们在 COCO数据集上验证了所提出的检测器，并采用相同的从零开始训练设置[21,65,62]。此外，所有模型的延迟均在T4 GPU上使用 TensorRT FP16进行测试，方法参照[78]。

4.2. 与最先进技术比较

如表 3.2-1所示，我们的YOLOv10在各类模型规模上实现了最先进的性能和端到端延迟。我们首先将YOLOv10与我们的基准模型（即YOLOv8）进行比较。在N/S/M/L/X五种变体上，我们的YOLOv10分别实现了1.2% / 1.4% / 0.5% / 0.3% / 0.5%的AP提升，参数量减少28%/36%/41%/44%/57%，计算量减少23%/24%/25%/27%/38%，延迟降低70%/65%/50%/41%/37%。与其他YOLO相比，YOLOv10在精度与计算成本之间也表现出更优的权衡。具体来说，对于轻量级和小型模型，YOLOv10-N/S比YOLOv6-3.0-N/S分别高出1.5 AP和2.0 AP，同时参数量少51%/61%，计算量减少41%/52%。对于中型模型，相比YOLOv9-C/YOLO-MS，YOLOv10-B/M在取得相同或更好性能的情况下，延迟分别降低46%/62%。对于大型模型，相比Gold-YOLO-L，我们的YOLOv10-L参数量减少68%，延迟降低32%，同时AP有显著提升1.4%。此外，与RT-DETR相比，YOLOv10在性能和延迟上都有显著提升。值得注意的是，在类似性能下，YOLOv10-S/X的推理速度分别比RT-DETR-R18/R101快1.8倍和1.3倍。这些结果充分展示了YOLOv10作为实时端到端检测器的优势。

我们还使用原始的一对多训练方法将 YOLOv10与其他 YOLO 模型进行了比较。在这种情况下，我们考虑模型前向过程的性能和延迟（Latency_f），参考[62,21,60]。如表 3.2-1所示，YOLOv10在不同模型规模上同样表现出最先进的性能和效率，这表明我们架构设计的有效性。

表 4.2-2 使用 YOLOv10-S 和 YOLOv10-M on COCO 的消融研究

# Model	NMS-free.	Efficiency.	Accuracy.	#Param.(M)	FLOPs(G)	$AP^{val}(\%)$	Latency(ms)
1				11.2	28.6	44.9	7.07
2 YOLOv10-S	✓			11.2	28.6	44.3	2.44
	✓	✓		6.2	20.8	44.5	2.31
4	✓	✓	✓	7.2	21.6	46.3	2.49
5				25.9	78.9	50.6	9.50
6	✓			25.9	78.9	50.3	5.22
7 YOLOv10-M	✓	✓		14.1	58.1	50.4	4.57
8	✓	✓	✓	15.4	59.1	51.1	4.74

表 4.2-3 双重分配

o2m	o2o	AP	Latency
✓		44.9	7.07
	✓	43.4	2.44
✓	✓	44.3	2.44

表 4.2-4 匹配度指标

α_{o2o}	β_{o2o}	AP^{val}	α_{o2o}	β_{o2o}	AP^{val}
0.5	2.0	42.7	0.25	3.0	44.3
0.5	4.0	44.2	0.25	6.0	43.5
0.5	6.0	44.3	1.0	6.0	43.9
0.5	44.3	44.0	1.0	12.0	44.3

表 4.2-5 效率。适用于 YOLOv10-S/M。

#Model	#Param	FLOPs	AP^{val}	Latency
1base.	11.2/25.9	28.6/78.9	44.3/50.3	2.44/5.22
2 +cls.	9.9/23.2	23.5/67.7	44.2/50.2	2.39/5.07
3 +downs.	8.0/19.7	22.2/65.0	44.4/50.4	2.36/4.97
4+block.	6.2/14.1	20.8/58.1	44.5/50.4	2.31/4.57

4.3. 模型分析

消融研究。我们基于YOLOv10-S和YOLOv10-M在表 4.2-2中展示了消融结果。可以观察到，我们采用一致双分配的无NMS训练显著降低了YOLOv10-S的端到端延迟4.63毫秒，同时保持了44.3%的AP的竞争性表现。此外，我们以效率为驱动的设计使参数减少了11.8M，计算量减少了20.8GFLOPs，并且YOLOv10-M的延迟显著降低了0.65毫秒，有效地展示了其效果。此外，我们以准确率为驱动的设计在YOLOv10-S和YOLOv10-M上分别实现了1.8 AP和0.7 AP的显著提升，同时仅增加了0.18毫秒和0.17毫秒的延迟开销，充分展示了其优越性。

4.3.1. 针对 NMS 的免费训练分析

双重标签分配。我们为无NMS的YOLO提出了双重标签分配，这既可以在训练过程中为一对多(o2m)分支提供丰富的监督，也可以在推理过程中为一对一(o2o)分支带来高效性。我们基于YOLOv8-S验证了其优势，即表 4.2-2中的排名第一。具体而言，我们分别引入了仅使用o2m分支和仅使用o2o分支的训练基线。如表 4.2-3所示，我们的双重标签分配实现了最佳的AP-延迟折中。

一致匹配度量。我们引入一致匹配度量，使一对一头与一对多头更加协调。我们基于YOLOv8-S，即表 4.2-2中的#1，在不同的 α_{o2o} 和 β_{o2o} 下验证了其效果。如表 4.2-4所示，所提出的一致匹配度量，即 $\alpha_{o2o} = r \cdot \alpha_{o2m}$ 和 $\beta_{o2o} = r \cdot \beta_{o2m}$ ，可以实现最佳性能，其中一对多头中的 $\alpha_{o2m} = 0.5$ 和 $\beta_{o2m} = 6.0$ 。这种改进可以归因于监督差距（公式 $A = t_{o2o,i} - I(i \in \Omega) t_{o2m,i} + \sum_{k \in \Omega \setminus \{i\}} t_{o2m,k}, (2)$ ）的减少，从而提供了两个分支之间更好的监督对齐。此外，所提出的一致匹配度量消除了对耗时的超参数调优的需求，这在实际场景中非常有吸引力。

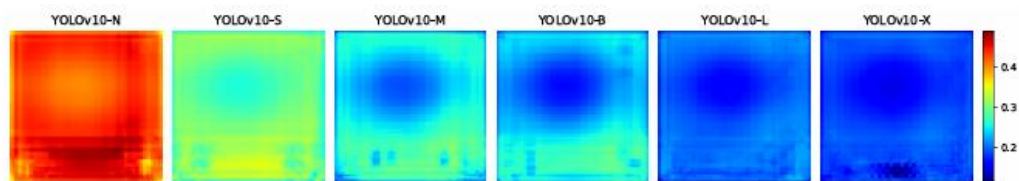


图 4.3.1-4 每个锚点提取特征与所有其他特征的平均余弦相似度

表 4.3.1-6 分类结果

	base.	+cls.
AP^{val}	44.3	44.2
$AP_{w/o c}^{val}$	59.9	59.9
$AP_{w/o r}^{val}$	64.5	64.2

表 4.3.1-7 d.s.的结果

Model	AP^{val}	Latency
base.	43.7	2.33
ours	44.4	2.36

表 4.3.1-8 CIB 的结果

Model	AP^{val}	Latency
IRB	43.7	2.30
IRB-DW	44.2	2.30
ours	44.5	2.31

表 4.3.1-9 按等级指导

Stages with CIB	AP^{val}
empty	44.4
8	44.5
8, 4,	44.5
8, 4, 7	44.3

与一对多训练相比的性能差距。虽然在无NMS训练下实现了更优的端到端性能，但我们观察到，与使用NMS的原始一对多训练相比，性能差距仍然存在，如表 4.2-3和表 3.2-1所示。此外，我们注意到，随着模型规模的增加，这种差距有所减小。因此，我们可以合理地得出结论，这种差距可以归因于模型能力的局限性。值得注意的是，与使用NMS的原始一对多训练不同，无NMS训练对于一对一匹配需要更具判别力的特征。在YOLOv10-N模型的情况下，其有限的能力导致提取的特征缺乏足够的判别力，从而导致1.0% AP的更明显性能差距。相比之下，具有更强能力和更具判别力特征的YOLOv10-X模型，在两种训练策略之间不存在性能差距。在图 4.3.1-4中，我们可视化了COCO验证集中每个锚点提取的特征与其他所有锚点特征的平均余弦相似度。我们观察到，随着模型规模的增加，锚点之间的特征相似度呈下降趋势，这有利于一对一匹配。基于这一见解，我们将在未来的工作中探索进一步缩小差距并实现更高端到端性能的方法。

4.3.2. 用于高效驱动模型设计分析

我们进行实验以逐步引入基于YOLOv10-S/M的效率驱动设计元素。我们的基线是没有效率-精度驱动模型设计的YOLOv10-S/M模型，即表 4.2-2中的#2/#6。如表 4.2-5所示，每个设计组件，包括轻量级分类头、空间-通道解耦下采样以及秩引导块设计，都有助于减少参数数量、FLOPs和延迟。重要的是，这些改进是在保持具有竞争力的性能的同时实现的。

轻量级分类头。我们分析了预测的类别和定位错误对性能的影响，基于表 4.2-5中的YOLOv10-Sof#1和#2，如[7]所示。具体来说，我们通过一对一分配将预测与实例匹配。然后，我们用实例标签替换预测的类别分数，从而得到没有分类错误的 $AP_{w/o.c}^{val}$ 。类似地，我们用实例的实际位置替换预测位置，从而得到没有回

归错误的 $AP_{w/o c}^{val}$ 。如表 4.3.1-6所示， $AP_{w/o r}^{val}$ 远高于 $AP_{w/o c}^{val}$ ，这表明消除回归错误带来的提升更大。因此，性能瓶颈更多地存在于回归任务中。因此，采用轻量级分类头可以在不影响性能的情况下实现更高的效率。

空间-通道解耦下采样。我们为提高效率而解耦下采样操作，先通过逐点卷积（PW）增加通道维度，然后通过深度卷积（DW）降低分辨率，以最大限度保留信息。我们将其与基线方式进行比较，即先通过DW进行空间降采样，再通过PW进行通道调整，基于YOLOv10-Sof#3的表 4.2-5。如表 4.3.1-7所示，我们的下采样策略在下采样过程中信息损失更少，实现了0.7%的AP提升。

紧凑型倒置块（CIB）。我们提出CIB作为紧凑的基础构建模块。我们基于表 4.2-5中的YOLOv10-Sof#4验证了其有效性。具体来说，我们引入倒置残差块（IRB）作为基线，其实现了次优的43.7%AP，如表 4.3.1-8所示。然后，我们在其后附加一个3×3的深度可分离卷积（DW），记作“IRB-DW”，带来了0.5%的AP提升。与“IRB-DW”相比，通过在前端再添加一个DW且开销最小，我们的CIB进一步实现了0.3%的AP提升，表明其优越性。

基于排序的模块设计。我们引入了基于排序的模块设计，以自适应地整合紧凑型模块设计，从而提高模型效率。我们基于表 4.2-5中#3的YOLOv10-S验证了其优点。根据内在秩升序排序的阶段为Stage 8-4-7-3-5-1-6-2，如表 4.2-3 (a)所示。如表 4.3.1-9所示，在逐步将各阶段的瓶颈模块替换为高效的CIB时，我们观察到性能从Stage 7开始下降。在Stage 8和4中，由于其内在秩较低且冗余较多，因此我们可以采用高效的模块设计而不影响性能。这些结果表明，基于排序的模块设计可以作为提高模型效率的有效策略。

表 4.3.2-10 精确度。适用于 S/M。

Model	AP^{val}	Latency
1 base.	44.5/50.4	2.31/4.57
2 +L.k.	44.9/-	2.34/-
3 +PSA	46.3/51.1	2.49/4.74

表 4.3.2-11 L.k. 结果

Model	AP^{val}	Latency
k. s. =5	44.7	2.32
k. s. =7	44.9	2.34
k. s. =9	44.9	2.37
w/o rep.	44.8	2.34

表 4.3.2-12 L.k. 用法。

	w/o L. k.	w/ L. k.
N	36.3	36.6
S	44.5	44.9
M	50.4	50.4

表 4.3.2-13 PSA 结果

Model	AP^{val}	Latency
PSA	46.3	2.49
Trans.	46.0	2.54
$N_{PSA} = 1$	46.3	2.49
$N_{PSA} = 2$	46.5	2.59

4.3.3. 用于精确驱动模型设计的分析

我们展示了基于YOLOv10-S/M逐步整合以精度为驱动的设计元素的结果。我们的基线是结合了以效率为驱动的设计后的 YOLOv10-S/M模型，即表 4.2-2中的 #3/#7。如表 4.3.2-10所示，采用大卷积核卷积和 PSA 模块使得YOLOv10-S的性能显著提升，AP分别提高了0.4%和1.4%，而延迟仅分别增加0.03ms和0.15ms。需要注意的是，YOLOv10-M并未使用大卷积核卷积（见表 4.3.2-12）。

大核卷积。我们首先基于表 4.3.2-10中的YOLOv10-S研究了不同卷积核大小的影响。如表 4.3.2-11所示，随着卷积核大小的增加，性能提高，并在7×7的卷积核大小附近趋于稳定，表明大感受野的优势。此外，在训练过程中移除重参数分支会导致AP下降0.1%，显示其对优化的有效性。此外，我们基于YOLOv10-N/S/M检查了大核卷积在不同模型规模下的效果。如表 4.3.2-12所示，对于大模型（即YOLOv10-M）没有带来提升，这是因为其固有的广泛感受野。因此，我们只在小模型（即YOLOv10-N/S）中采用大核卷积。

部分自注意力（PSA）。我们引入 PSA 以在最小成本下通过整合全局建模能力来提升性能。我们首先基于表 4.3.2-10中#3的YOLOv10 S验证其有效性。具体来说，我们引入了Transformer模块，即先MHSA再FFN，作为基线，记作“Trans.”。如表 4.3.2-13所示，与其相比，PSA 带来了0.3%的AP提升，同时延迟降低了0.05ms。性能提升可能归因于缓解了自注意力中的优化问题，通过减少注意力头的冗余。此外，我们还研究了不同 NPSA 的影响。如表 4.3.2-13所示，将NPSA增加到2可获得0.2%的AP提升，但会带来0.1ms的延迟开销。因此，我们默认将NPSA设置为1，以在保持高效率的同时增强模型能力。

5. 结论

在本文中，我们针对YOLO检测流程中的后处理和模型架构进行了优化。在后处理方面，我们提出了一种一致的双重分配方法，用于无NMS训练，实现高效的端到端检测。在模型架构方面，我们引入了整体的效率-精度驱动的设计策略，提升性能与效率的权衡。这些改进带来了我们的YOLOv10，一种新的实时端到端目标检测器。大量实验表明，与其他先进检测器相比，YOLOv10在性能和延迟方面都达到了最先进水平，充分展示了其优越性。

参考文献

- [1]. Wang A , Chen H , Liu L ,et al.YOLOv10: Real-Time End-to-End Object Detection[J]. 2024.